

Rakugrid

A behavioural test suite for Raku, organised as a grid.

571 atoms

102,809 tests

1,812 signed rulings

runs on **any** implementation

tests **any** implementation

phase 0

— WHAT A CONFORMANCE PASS STILL LEAVES OPEN

90% of Roast — and still many corners to light.

Raku++ passed the large majority of Roast's ~216,000 assertions, and running ordinary Raku programs still turned up plenty to improve. Nearly every one of those findings came from somewhere **other than Roast**: writing example programs, running a corpus of real code, installing ecosystem modules, generating documentation sites.

ROAST

Is this Raku?

The official specification suite — 1,462 files, ~216,000 assertions. It defines the language, test by test.

RAKUGRID

Does it hold together?

Whether an implementation **survives contact with real Raku programs**. Built from what those findings taught us.

Not a replacement for Roast, and not a fork of it. Roast defines the language; Rakugrid measures whether an implementation of it holds together.

— THREE THINGS A SPECIFICATION SUITE IS NOT FOR

Provenance. Exhaustiveness. Combinations.

01 · PROVENANCE

● Every test says why

A bug we fixed, a corpus program that broke, a module that would not install, a documented rule. Nothing is here because it seemed like a good test — `from:` is a required field.

02 · EXHAUSTIVENESS

● Holes are visible

Coverage is measured against a machine-extracted inventory of the language: 182 operators, 198 subroutines, 667 methods, ~110 types. An atom with zero tests is a **visible hole**, not an unknown unknown.

03 · COMBINATIONS

● Crossings, systematically

Most hard bugs were constructs that each worked alone and broke together — `return` inside `CATCH`, `next` inside `.map` inside `for`, a dynamic variable crossing a regex subrule boundary.

Each of those bugs needed **two or three constructs interacting**, and none is reachable by testing either construct alone. That is the layer Rakugrid adds.

— THREE WORDS, AND THE WHOLE SUITE FOLLOWS FROM THEM

Atoms hold the line. Molecules find the bugs.

the **grid** – eight orthogonal facet axes

atom

One construct exercising one behaviour in isolation. A point in the grid.

atoms prevent regressions

molecule

Constructs in combination, where each part is known to work alone. A covering array over the axes.

molecules find bugs

ladder

The corner values a position accepts — `Inf`, `NaN`, `-0e0`, empty, the bignum boundary, the type object.

what flows through the point

Identity is stable and citable: `grid:operators/infix-plus#0041` — the atom, then a four-digit number unique within it, **never reused and never renumbered**. When a molecule fails, the shrinker walks the facet axes to the minimal failing coordinate; the result is a valid minimal repro and is promoted into `atoms/` with its provenance intact.

— LEVELS EXTRACTED FROM THE GRAMMAR AND FROM OUR OWN BUG HISTORY

Eight axes. ~74 million points.

nesting

sub · method · bare block · loop body · do · gather · start · CATCH
· phaser · regex code block · given/when · thunk · react/whenever

container

scalar · array · hash · bound := · native-typed · readonly ·
itemized · attribute · dynamic \$* · state · constant

evaluation

eager · lazy · infinite · gathered · hyper · threaded · precompiled

scope

file · block · closure · role · class · module · EVAL · precompiled
module

exit

fall-through · return · last · next · redo · die · take · emit ·
succeed · fail · warn

context

item · list · sink · Bool · Str · Num · Int · slurpy argument ·
hash key

dispatch

sub · multi · method · private · meta-operator · operator · &f
reference · .& · indirect

type shape

definite · type object · subset · coercion · but mixin · junction ·
Failure · Nil

The full cartesian product is ~74 million points before payloads — which is why this is generated as a **covering array at a chosen interaction strength** rather than enumerated. Empirical fault studies find essentially all defects triggered by six or fewer interacting factors, with 2-way already catching the large majority. Strength is a dial, not a constant.

— FACETS SAY WHERE A CONSTRUCT SITS — LADDERS SAY WHAT FLOWS THROUGH IT

Corner against corner, in both directions.

Int $0 \cdot 1 \cdot -1 \cdot 2 \cdot 2^{63} \cdot -1 \cdot 2^{64} \cdot -2^{64}$ — the bignum boundary both ways

Num $0e0 \cdot -0e0 \cdot 1e0 \cdot \text{Inf} \cdot -\text{Inf} \cdot \text{NaN} \cdot \text{a denormal} \cdot \text{max finite}$

Rat $0/1 \cdot 1/1 \cdot 1/3 \cdot \text{a denominator past uint64}$ — the $\text{Rat} \rightarrow \text{Num}$ boundary

Str `""` · `" "` · `"0"` · a multi-codepoint grapheme · an embedded NUL · a very long string

List `()` · `(1,)` · nested · lazy · infinite · self-referential

Range `empty` `(5..1)` · `1..Inf` · `-Inf..Inf` · reversed · non-numeric

undefined the type object · Any · Mu · Nil · an unhandled Failure

An atom is not covered because it has tests — it is covered when every position's ladder is complete and the legal crosses are exercised in **both orders**. Order matters even for operators that ought to commute, because "ought to" is precisely the thing under test. Real defects sit exactly there: `Inf + -Inf` must be `NaN` and not `0`; `-0e0` must survive an addition; a `Rat` whose denominator crosses `uint64` must degrade to `Num` rather than wrap — a defect Raku++ actually shipped.

Every operator, on ordinary values first.

one atom
per
operator

OPERATOR ↓
OPERANDS →

INT × INT

1 · -1

INT × RAT

1 · 1/2

STR × STR

"a" · "0"

LIST × RANGE

(1,2) · (1..3)

	INT × INT	INT × RAT	STR × STR	LIST × RANGE
+	(1) + (-1) → 0 Int	(1) + (1/2) → 1.5 Rat	("a") + ("0") → X::Str::Numeric	((1,2)) + ((1..3)) → 5 Int
*	(1) * (-1) → -1 Int	(1) * (1/2) → 0.5 Rat	("a") * ("0") → X::Str::Numeric	((1,2)) * ((1..3)) → 6 Int
**	(1) ** (-1) → 1.0 Rat	(1) ** (1/2) → 1e0 Num	("a") ** ("0") → X::Str::Numeric	((1,2)) ** ((1..3)) → 8 Int
~	(1) ~ (-1) → "1-1" Str	(1) ~ (1/2) → "10.5" Str	("a") ~ ("0") → "a0" Str	((1,2)) ~ ((1..3)) → "1 21 2 3" Str
x	(1) x (-1) → "" Str	(1) x (1/2) → "" Str	("a") x ("0") → "" Str	((1,2)) x ((1..3)) → "1 21 21 2" Str
==	(1) == (-1) → Bool::False Bool	(1) == (1/2) → Bool::False Bool	("a") == ("0") → X::Str::Numeric	((1,2)) == ((1..3)) → Bool::False Bool
eq	(1) eq (-1) → Bool::False Bool	(1) eq (1/2) → Bool::False Bool	("a") eq ("0") → Bool::False Bool	((1,2)) eq ((1..3)) → Bool::False Bool
<=>	(1) <=> (-1) → Order::More Order	(1) <=> (1/2) → Order::More Order	("a") <=> ("0") → X::Str::Numeric	((1,2)) <=> ((1..3)) → Order::Less Order
cmp	(1) cmp (-1) → Order::More Order	(1) cmp (1/2) → Order::More Order	("a") cmp ("0") → Order::More Order	((1,2)) cmp ((1..3)) → Order::Less Order
min	(1) min (-1) → -1 Int	(1) min (1/2) → 0.5 Rat	("a") min ("0") → "0" Str	((1,2)) min ((1..3)) → \$(1, 2) List

The generator reads the operator list out of the documentation and crosses each one with a mixed ladder that is **mostly everyday values** — 0, 1, -1, 1/2, "a", True, (1,2) — with the corner cases inside the *same* cross, as a minority. **139 operator atoms, 69,576 tests**, each asserting the value *and* its type.

Two ladder values make an **expression**.

$(\boxed{A}) + (\boxed{B})$ A ↓ B →	RAT 1/3	INT 2**64	NUM -0e0	NUM Inf	NUM -Inf
RAT 1/3	$(1/3) + (1/3)$ → <2/3> Rat	$(1/3) + (2**64)$ → <55340232221128654 849/3> Rat	$(1/3) + (-0e0)$ → 0.3333333333333333 e0 Num	$(1/3) + (Inf)$ → Inf Num	$(1/3) + (-Inf)$ → -Inf Num
INT 2**64	$(2**64) + (1/3)$ → <55340232221128654 849/3> Rat	$(2**64) + (2**64)$ → 368934881474191032 32 Int	$(2**64) + (-0e0)$ → 1.8446744073709552 e+19 Num	$(2**64) + (Inf)$ → Inf Num	$(2**64) + (-Inf)$ → -Inf Num
NUM -0e0	$(-0e0) + (1/3)$ → 0.3333333333333333 e0 Num	$(-0e0) + (2**64)$ → 1.8446744073709552 e+19 Num	$(-0e0) + (-0e0)$ → -0e0 Num	$(-0e0) + (Inf)$ → Inf Num	$(-0e0) + (-Inf)$ → -Inf Num
NUM Inf	$(Inf) + (1/3)$ → Inf Num	$(Inf) + (2**64)$ → Inf Num	$(Inf) + (-0e0)$ → Inf Num	$(Inf) + (Inf)$ → Inf Num	$(Inf) + (-Inf)$ → NaN Num
NUM -Inf	$(-Inf) + (1/3)$ → -Inf Num	$(-Inf) + (2**64)$ → -Inf Num	$(-Inf) + (-0e0)$ → -Inf Num	$(-Inf) + (Inf)$ → NaN Num	$(-Inf) + (-Inf)$ → -Inf Num

The atom carries a form with two holes and a ladder for each; the generator fills them and records the value *and* the type.

Exactness survives the bignum boundary — `1/3 + 2**64` is still an exact `Rat`, `2**64 + 2**64` an `Int` that does not wrap — while a single `Num` operand degrades the whole expression, and `-0e0` survives being added to itself. `Inf + -Inf` is `NaN` in **both** orders, which is exactly why the ladder crosses both. This one operator is $11 \times 11 = 121$ **expressions**.

Two atoms make a **program**.

NESTING SKELETON ↓
EXIT PAYLOAD →

RETURN
return "R"

LAST
last

TAKE
take "T"

SUB-BODY

```
do { sub f() { ; "fell" };  
f() }
```

```
do { sub f() { return "R"; "fell"  
}; f() }  
→ "R"
```

```
do { sub f() { last; "fell" }; f()  
}  
→ X::ControlFlow
```

```
do { sub f() { take "T"; "fell" };  
f() }  
→ X::ControlFlow
```

BARE-BLOCK

```
do { ; "fell" }
```

```
do { return "R"; "fell" }  
→ X::ControlFlow::Return
```

```
do { last; "fell" }  
→ X::ControlFlow
```

```
do { take "T"; "fell" }  
→ X::ControlFlow
```

LOOP-IN-SUB

```
do { sub f() { for 1..3 {  }  
}; "end" }; f() }
```

```
do { sub f() { for 1..3 { return  
"R" }; "end" }; f() }  
→ "R"
```

```
do { sub f() { for 1..3 { last };  
"end" }; f() }  
→ "end"
```

```
do { sub f() { for 1..3 { take "T"  
}; "end" }; f() }  
→ X::ControlFlow
```

GATHER

```
do { gather { ; take  
"after" }.list.raku }
```

```
do { gather { return "R"; take  
"after" }.list.raku }  
→ X::ControlFlow::Return
```

```
do { gather { last; take "after"  
}.list.raku }  
→ "()"
```

```
do { gather { take "T"; take  
"after" }.list.raku }  
→ "(\\"T\\", \\"after\\")"
```

A skeleton is an AST-shaped template with a hole; the payload goes in the hole, and the pair *is* the test. Composition guarantees the program is well-formed — and because the rule is known, the expected result can be **derived** rather than merely observed; where the oracle disagrees with the derivation, that is a finding. Same payload, different enclosure, different answer: `last` throws in a sub, ends the loop in a loop, and empties the sequence in a `gather`. The full crossing is 14 nesting levels × 10 exits = **140 programs**, 46 of them carrying a divergence.

— RAKUDO IS THE ORACLE, BUT NOT THE ARBITER

Divergence is always possible. **Never silent.**

Every test stores both what the reference actually did (`oracle:`) and what we assert (`expect:`). Usually identical. Where they differ, the test carries a signed **verdict** with a rationale and a date — and a divergence with no verdict **fails the build**.

docs

documentation is explicit and the implementation contradicts it

reason

neither docs nor implementation settle it; the ruling follows from the language's own consistency

impl-bug

we believe it is a defect; links the upstream issue

unanswerable

the implementation yields no observation at all — hang, OOM, crash

underspecified

more than one answer is legitimate; assert only the property they share

disputed

unresolved; the test is parked and does not run

Standing rulings settle recurring questions once: IEEE-754 beats the reference (correctly-rounded values derived from 80-digit arithmetic, never from another libm); diagnostic wording is never asserted, only the exception type; unspecified order — hash and set iteration — is never asserted at all.

One file per atom. One behaviour per record.

```
atom    hangs/str-encode-invalid-argument
source  curated
kind    termination – the assertion is that the program ends

– id    0001
  from  generator:gen/methods.raku · review:2026-08-15
  code  """.encode(0)
  finishes 5s
  oracle rakudo-2026.07 → TIMEOUT after 6s
  oracle rakupp-3.14.0 → utf8:0x<>
  verdict impl-bug
  why    an encoding name that is not a string must fail, and no argument makes
         non-termination correct – one bad cell stalled the method matrix 87 min
  ruled  2026-08-15 against rakudo-2026.07
```

EXACTLY ONE ASSERTION PER RECORD

is · type · throws · output

Plus `finishes`, `same-as`, `parses`, `no-parse`. The first four run in-process, many to a program; the last four need a process each, because a parse failure or a hang would take every other test in the file down with it.

HOW AN EXPECTED VALUE IS PRODUCED

Never written by a person

An `is` value is whatever the reference's `.raku` printed, copied verbatim — and compared **as text**. Text is what makes `NaN` equal `NaN`, keeps `-0e0` distinct from `0e0`, and survives values whose `.raku` the engine then refuses to re-compile.

Seven generators, one command to re-oracle.

<code>ladder.raku</code>	a hand-written table of 53 operators and methods crossed with their edge ladders — value assertions per cell
<code>operators.raku</code>	the documented inventory — 125 infix, 18 prefix, 2 postfix — every one crossed with a ladder. An operator the language documents and nobody tested becomes impossible, not merely unlikely
<code>methods.raku</code>	asks the implementation what a type has — .^methods — and crosses each method with that type's ladder. One atom per type, so a hole is visible at a glance
<code>regex.raku</code>	constructs $\times$ subjects $\times$ match forms, asserting derived facts — so(\$s ~ /p/) , .Str , .from — never Match internals, which differ for reasons that are not about matching
<code>syntax.raku</code>	declarators, signatures, phasers, quoting — compiled with -c and never run: asserts only whether it compiles , never the wording of the diagnostic
<code>laws.raku</code>	algebraic laws over ladders as same-as — no oracle at all . An engine that disagrees with itself is wrong without anyone deciding what the right answer was
<code>molecules.raku</code>	two facet axes crossed — the interaction layer, one atom per crossing

Plus `import-regression.raku`, which imports whole regression programs with their fixtures and provenance. Ladders are **append-only** — ids come from a cell's position and are permanent — so widening one costs only the new cells.

— EVERY ITEM HERE EXISTS BECAUSE IT WENT WRONG — NONE IS HYPOTHETICAL

Probing a language is **hostile terrain**.

- ▶ **Parenthesise the operands.** `1/2 ** 2` parses as `1/(2**2)` — without parens a cell silently tests something other than what its label says.
- ▶ **Confirm compile failures against a real file.** EVAL is not a file: Rakudo compiles `sub { (0) ~~ (Nil) }` happily from a file and dies on the identical text under EVAL. Of 38 flagged compile failures, **28 were EVAL artefacts**.
- ▶ **Compile-check in a different process.** Compiling `sub { EXPR }` *declares* whatever EXPR declares, so every class-bearing cell reported a redeclaration that existed only inside the probe.
- ▶ **Give every cell its own names.** Ten cells declaring `class C` are ten redeclarations once packed into one dense program. Names are derived from the cell's coordinates.
- ▶ **Probe the reference twice.** `.WHICH` on a reference type carries an address — a test built from one run is flaky by construction. Any cell answering differently on two identical runs is parked.
- ▶ **Never assert a non-answer.** `CRASH:1` was once written into an `is` field. A reference that crashed or hung gives us nothing to assert.
- ▶ **Nothing dangerous, ever.** Only value types are introspected, and anything that could reach the outside world is refused by name. A generated suite must never be able to delete a file because some type grew an `.unlink`.

— PRINTED BY RAKUGRID STATS , NEVER HAND-MAINTAINED

102,809 tests, all of them traceable.

571 atoms

102,539 generated + 270 curated, across 605 .grid files.

21,639

observations where the recorded engines disagree — clustered into **3,683** distinct divergences, plus **113 crashes** on record.

1,812 rulings

signed verdicts where expect diverges from oracle. An unsigned one fails the build.

inventory		69,576
types		24,724
operators		3,845
laws		3,051
methods		705
syntax		434
curated		270
molecules		204

Two engines on record throughout — [rakudo-2026.07](#) and [rakupp-3.14.0](#). Curated atoms are the smallest family and the most expensive: each one is a real bug, minimised, with the story of how it broke written into the record.

Density is a dial. ~580k at the top of it.

L0 ~3k	seconds — every build; touches every atom once
L1 ~30k	under a minute — the commit gate
L2 ~150k	a few minutes — pre-push
L3 ~580k	nightly — the release gate. Roughly 2.7× Roast's declared assertions, every test traceable to a reason
L4 past 1M	on demand — 4-wise on core atoms, or one atom exhaustively. A million reached through interaction depth , not per-codepoint padding

- ▶ **Not yet built:** the atom and facet inventory extractors, ladder-completeness in cover, directed shrinking, and the remaining facet axes.
- ▶ **Ahead:** the Roast harvest — take what Roast *covers*, never what it *contains*: parse its 1,462 files, extract every assertion as a triple, emit a coverage map, and generate only where a shape is uncovered. Expected values always come from a fresh oracle run, never from Roast's literal.
- ▶ **Then:** corpus and module-battery mining, the illegality grid, and holes closed to zero — followed by a CI matrix across implementations.

— THE HARNESS RUNS ON ANY IMPLEMENTATION AND TESTS ANY IMPLEMENTATION

Point it at an **engine**.

```
$ rakugrid fire # run everything, emit TAP
```

```
$ rakugrid fire --engine=/path/to/engine # ...against another one
```

```
$ rakugrid check # the build rule: no unsigned divergences
```

```
$ rakugrid diverge # where recorded engines disagree, clustered
```

```
$ rakugrid isolate grid:operators/infix-plus#0041 # any test, standalone
```

The two need not be the same implementation. `rakupp bin/rakugrid fire` tests Rakup++, `raku bin/rakugrid fire` tests Rakudo, and `--engine=` overrides either. Nothing in the suite requires a particular implementation to be present — Rakudo is consulted for one thing only: knowing what a piece of code does.

github.com/ash/rakugrid

github.com/ash/rakupp

raku.online/rules

[Raku/roast](https://Raku.roast)